

## SECURITY AND DATA

# Security

Learn about Claude Code's security safeguards and best practices for safe usage.

## How we approach security

### Security foundation

Your code's security is paramount. Claude Code is built with security at its core, developed according to Anthropic's comprehensive security program. Learn more and access resources (SOC 2 Type 2 report, ISO 27001 certificate, etc.) at [Anthropic Trust Center](#).

### Permission-based architecture

Claude Code uses strict read-only permissions by default. When additional actions are needed (editing files, running tests, executing commands), Claude Code requests explicit permission. Users control whether to approve actions once or allow them automatically.

Claude Code requires approval before running Bash commands that can modify your system. A built-in set of read-only commands such as `ls`, `cat`, and `git status` runs without a prompt. This approach lets users and organizations configure permissions directly.

For detailed permission configuration, see [Permissions](#).

### Built-in protections

To mitigate risks in agentic systems:

- **Sandboxed bash tool:** [Sandbox](#) bash commands with filesystem and network isolation, reducing permission prompts while maintaining security. Enable with `/sandbox` to define boundaries where Claude Code can work autonomously
- **Write access restriction:** Claude Code can only write to the folder where it was started and its subfolders—it cannot modify files in parent directories without explicit permission. While Claude Code can read files outside the working directory (useful for accessing system libraries

and dependencies), write operations are strictly confined to the project scope, creating a clear security boundary

- **Prompt fatigue mitigation:** Support for allowlisting frequently used safe commands per-user, per-codebase, or per-organization
- **Accept Edits mode:** Auto-approves file edits and a fixed set of filesystem Bash commands like `mkdir`, `touch`, `rm`, `mv`, `cp`, and `sed` for paths in the working directory. Other Bash commands and out-of-scope paths still prompt

## User responsibility

Claude Code only has the permissions you grant it. You're responsible for reviewing proposed code and commands for safety before approval.

## Protect against prompt injection

Prompt injection is a technique where an attacker attempts to override or manipulate an AI assistant's instructions by inserting malicious text. Claude Code includes several safeguards against these attacks:

### Core protections

- **Permission system:** Sensitive operations require explicit approval
- **Context-aware analysis:** Detects potentially harmful instructions by analyzing the full request
- **Input sanitization:** Prevents command injection by processing user inputs
- **Network command approval:** Commands that fetch content from the web such as `curl` and `wget` are not auto-approved by default. They prompt like any other non-read-only Bash command, so you can still approve once or add an explicit allow rule like `Bash(curl *)`. To block them entirely, add them to `permissions.deny`

## Privacy safeguards

We have implemented several safeguards to protect your data, including:

- Limited retention periods for sensitive information (see the [Privacy Center](#) to learn more)
- Restricted access to user session data
- User control over data training preferences. Consumer users can change their [privacy settings](#)

at any time.

For full details, please review our [Commercial Terms of Service](#) (for Team, Enterprise, and API users) or [Consumer Terms](#) (for Free, Pro, and Max users) and [Privacy Policy](#).

## Additional safeguards

- **Network request approval:** Tools that make network requests require user approval by default
- **Isolated context windows:** Web fetch uses a separate context window to avoid injecting potentially malicious prompts
- **Trust verification:** First-time codebase runs and new MCP servers require trust verification
  - Note: Trust verification is disabled when running non-interactively with the `-p` flag
  - Note: When you start Claude Code directly in your home directory, trust acceptance is held for the current session only and is not written to disk, so the prompt reappears on each launch. There is no setting to persist it. Start Claude Code from a project subdirectory instead, where trust acceptance is saved per directory
- **Command injection detection:** Suspicious bash commands require manual approval even if previously allowlisted
- **Fail-closed matching:** Unmatched commands default to requiring manual approval
- **Natural language descriptions:** Complex bash commands include explanations for user understanding
- **Secure credential storage:** API keys and tokens are stored in the macOS Keychain when available, and protected by file permissions on Windows and Linux. See [Credential Management](#)

 **Windows WebDAV security risk:** When running Claude Code on Windows, we recommend against enabling WebDAV or allowing Claude Code to access paths such as `\\*` that may contain WebDAV subdirectories. **WebDAV has been deprecated by Microsoft** due to security risks. Enabling WebDAV may allow Claude Code to trigger network requests to remote hosts, bypassing the permission system.

## Best practices for working with untrusted content:

1. Review suggested commands before approval
2. Avoid piping untrusted content directly to Claude
3. Verify proposed changes to critical files
4. Use virtual machines (VMs) to run scripts and make tool calls, especially when interacting with external web services

## 5. Report suspicious behavior with [/feedback](#)

⚠ While these protections significantly reduce risk, no system is completely immune to all attacks. Always maintain good security practices when working with any AI tool.

## MCP security

Claude Code allows users to configure Model Context Protocol (MCP) servers. The list of allowed MCP servers is configured in your source code, as part of Claude Code settings engineers check into source control.

We encourage either writing your own MCP servers or using MCP servers from providers that you trust. You are able to configure Claude Code permissions for MCP servers. Anthropic reviews connectors against its [listing criteria](#) before adding them to the [Anthropic Directory](#), but does not security-audit or manage any MCP server.

## IDE security

See [VS Code security and privacy](#) for more information on running Claude Code in an IDE.

## Cloud execution security

When using [Claude Code on the web](#), additional security controls are in place:

- **Isolated virtual machines:** Each cloud session runs in an isolated, Anthropic-managed VM
- **Network access controls:** Network access is limited by default and can be configured to be disabled or allow only specific domains
- **Credential protection:** Authentication is handled through a secure proxy that uses a scoped credential inside the sandbox, which is then translated to your actual GitHub authentication token
- **Branch restrictions:** Git push operations are restricted to the current working branch
- **Audit logging:** All operations in cloud environments are logged for compliance and audit purposes
- **Automatic cleanup:** Cloud environments are automatically terminated after session completion

For more details on cloud execution, see [Claude Code on the web](#).

**Remote Control** sessions work differently: the web interface connects to a Claude Code process running on your local machine. All code execution and file access stays local, and the same data that flows during any local Claude Code session travels through the Anthropic API over TLS. No cloud VMs or sandboxing are involved. The connection uses multiple short-lived, narrowly scoped credentials, each limited to a specific purpose and expiring independently, to limit the blast radius of any single compromised credential.

## Security best practices

### Working with sensitive code

- Review all suggested changes before approval
- Use project-specific permission settings for sensitive repositories
- Consider using **dev containers** for additional isolation
- Regularly audit your permission settings with `/permissions`

### Team security

- Use **managed settings** to enforce organizational standards
- Share approved permission configurations through version control
- Train team members on security best practices
- Monitor Claude Code usage through **OpenTelemetry metrics**
- Audit or block settings changes during sessions with **ConfigChange hooks**

## Reporting security issues

If you discover a security vulnerability in Claude Code:

1. Do not disclose it publicly
2. Report it through our **HackerOne program**
3. Include detailed reproduction steps
4. Allow time for us to address the issue before public disclosure

## Related resources

- **Security guidance plugin**: have Claude review and fix vulnerabilities in its own code changes during the session
- **Sandbox environments**: compare isolation approaches and choose one for your threat model
- **Sandboxing**: filesystem and network isolation for Bash commands
- **Permissions**: configure permissions and access controls
- **Monitoring usage**: track and audit Claude Code activity
- **Development containers**: secure, isolated environments
- **Anthropic Trust Center**: security certifications and compliance